Reading an entire file in one go is fine, but might pose problems if the file concerned is particularly large. As we've seen already in the sections concerning configuration, PHP has a cap on how much memory it is allowed to allocate. Rather than dump the entire file into memory in one go, it is also possible to read and process a little at a time. Here's how to open a file and read a line at a time:

```
$handle = fopen(__DIR__ . '/data/sample.txt', 'r');
print fgets($handle);
print fgets($handle);
fclose($handle);
```

```
Output:
Star Trek: Enterprise was underrated.
The Xindi were better antagonists than the Suliban.
```

The function `fopen()` creates a file handle, a special type of resource variable that binds to the underlying data source (in this case the file) and is used by other functions to access that data. The first argument is the path to the file; the second is a mode parameter: `'r'` for read-only (other modes include `'r+'` for read/write, `'w'` for write-only, and truncation to zero bytes). `fgets()` streams each byte of the file until it hits a newline and returns those bytes as a single PHP string. It is good practice to call `fclose()` on file handles that are no longer required because they consume operating system resources (file descriptors).

There's a variety of similar functions to `fgets()` that operate on a file handle resource, summarized below:

Table 1-5. *Read functions*

`fgetss()`	Get a line and strip HTML tags
`fgetc()`	Get a single character
`fgetcsv()`	Get a field from a delimited file format
`fread()`	Binary-safe file access (see Chapter 8, "Strings," for explanation of binary-safety)